# 智能餐盒核心 API 接口规范 (v1.1)

**Base URL**: `/api/v1` （API地址域名待定，目前先确定一下整体）

**Content-Type**: `application/json`

## 1. 硬件遥测数据上报 (Telemetry Ingestion)

处理底层硬件高频上报的原始传感器数据，驱动后端有限状态机 (FSM) 流转。

- **接口路径**: `POST /hardware/telemetry`
- **调用频率**: 硬件端 5000ms/次 定时轮询上报。
- **Request Body**:
- JSON

代码块

```
{
  "device_id": "string",  // 硬件 MAC 地址或唯一标识"timestamp": 1715000000, // 硬件本地 Unix 时间戳 (秒级，绝对时间，无时区概念，默认 UTC)"weights": {
    "grid_1": 150.5, // 物理重量 (单位: 克/g，对应餐盒左上角主食分格)"grid_2": 80.0,  // 物理重量 (单位: 克/g，对应右上角菜品分格)"grid_3": 120.0, // 物理重量 (单位: 克/g，对应左下角菜品分格)"grid_4": 50.0   // 物理重量 (单位: 克/g，对应右下角汤/小菜分格)
  }
}
```

- **后端状态机跃迁逻辑 (具象化阈值)**:
  a. **死区拦截 (Deadband)**: 若当前处于 `IDLE` 且总重量变化量 $|\Delta W| < 5.0g$，直接抛弃 Payload。
  b. **备餐判定 ( `IDLE` $\rightarrow$ `SERVING` )**: 若 $|\Delta W| > 50.0g$ 且时间窗口 $< 60s$。
  c. **就餐判定 ( `SERVING` $\rightarrow$ `EATING` )**: 重量一阶导数 $\frac{dW}{dt} \le 0$ 并持续 $15s$。
  d. **结算判定 ( `EATING` $\rightarrow$ `IDLE` )**: 绝对重量 $< 10.0g$ 或连续 $600s$ 内 $|\Delta W| < 1.0g$。
- **Response**: `200 OK` (不返回业务数据，仅作为 ACK 确认，降低带宽开销)

## 2. 获取历史用餐记录列表 (Meal List - 游标分页版)

获取当前用户的历史就餐元数据。**为保障千万级数据下的查询性能，彻底废弃 Limit/Offset，改用游标分页 (Cursor-based Pagination)。**

- **接口路径**: `GET /meals`
- **Query Parameters**:
  - `user_id` (string, required)
  - `cursor` (integer, optional): 上一次查询返回的最后一条记录的 `start_time` 时间戳。首次查询不传。
  - `limit` (integer, optional, default: 20): 本次拉取的最大条数。
- **Response (200 OK)**:
- JSON

代码块

```
{
  "data": [
    {
      "meal_id": "string",
      "start_time": 1715000000,
      "total_served_g": 450.5
    }
  ],
  "pagination": {
    "next_cursor": 1714900000, // 用于下一次请求的游标"has_more": true // 客户端依据此字段判断是否显示"加载更多"// 移除 total 字段，避免 COUNT(*) 扫表带来的严重性能损耗
  }
}
```

## 3. 获取单次用餐详情 (Meal Detail)

获取指定 `meal_id` 的聚合统计数据。

- **性能保障声明**: 本接口**绝对不参与**任何实时计算。所有聚合计算（摄入量、时长等）均在状态机触发 `EATING` $\rightarrow$ `IDLE` 时，由后端异步 Worker 计算完毕并持久化至主表。本接口仅执行 $O(1)$ 级别的主键读取。
- **接口路径**: `GET /meals/{meal_id}`
- **Response (200 OK)**:
- JSON

代码块

```
{
  "meal_id": "string",
  "duration_minutes": 25,
  "total_served_g": 400.5,
  "total_leftover_g": 50.0,
  "total_intake_g": 350.5
}
```

## 4. 获取就餐时序轨迹 (Meal Trajectory - 动态降采样版)

用于渲染客户端 ECharts 进度折线图。支持全局拉取、增量轮询与动态降采样。

**接口核心设计说明**：本接口采用“大一统”架构设计，通过极其轻量级的参数控制，**同时兼顾“历史全量渲染”与“实时增量追加”两种业务场景**，前后端均无需维护两套独立的拉取/推送逻辑。

- **接口路径**: `GET /meals/{meal_id}/trajectory`
- **Query Parameters**:
  - `last_timestamp` (integer, optional): **【核心业务开关】** 增量查询的基准时间戳（开区间）。
  - `sample_interval` (integer, optional): 动态降采样间隔（秒）。例如传 `60`，后端将每 60 秒的数据求均值后返回，防止深历史数据导致的客户端 OOM (内存溢出) 和渲染卡顿。

---

### 📌 场景 A：拉取历史全量曲线 (Historical Full-Curve)

- **业务触发**: 用户在 App 历史列表中点击某顿已结束的饭，或需要初始化实时的首屏折线图。
- **前端传参规则**: **严格禁止传递 `last_timestamp` 参数。**
- **Request 示例**: `GET /meals/meal_20260311_1230/trajectory?sample_interval=30`
- **后端流转逻辑**:
- 识别到无 `last_timestamp`，直接执行 `SELECT timestamp, weights FROM meal_curve_data WHERE meal_id = ? ORDER BY timestamp ASC`，下发该顿饭从头到尾的 100% 全量切片数据（若有降采样参数则执行聚合算法）。

### 📌 场景 B：实时就餐进度追加 (Real-time Incremental Polling)

- **业务触发**: 用户正在就餐，App 屏幕停留在图表页，需要让折线图呈现“贪吃蛇”般的实时向右生长动画。
- **前端传参规则**: 前端每隔 5 秒发起轮询，**必须携带本地数组中最后一个（最新）点的时间戳作为 `last_timestamp`**。
- **Request 示例**: `GET /meals/meal_20260311_1230/trajectory?last_timestamp=1715000050`

- **后端流转逻辑**:
- 识别到增量游标，执行 `SELECT timestamp, weights FROM meal_curve_data WHERE meal_id = ? AND timestamp > 1715000050 ORDER BY timestamp ASC`。通常情况下，响应体中只包含这 5 秒内新增的 1 个点（甚至为空）。前端拿到后调用 `ECharts.appendData()` 进行局部重绘。

---

- **Response (200 OK) - 数据结构通用格式**:
- JSON

代码块

```
{
  "meal_id": "string",
  "query_mode": "historical", // 后端可显式告知前端当前触发的是 full 还是 incremental"points": [
    {
      "timestamp": 1715000000,
      "weights": {
        "grid_1": 150.0,
        "grid_2": 80.0,
        "grid_3": 120.0,
        "grid_4": 50.0
      }
    }
    // 场景A可能返回 300 个点；场景B通常只返回 1-2 个点
  ]
}
```